

Documentación Oficial de Jules

Módulo 2: Referencia de la REST API

Google Jules Agent Docs

Fuente: <https://jules.google/docs>

Tabla de Contenido

1. [Quickstart](#)
2. [API Reference](#)
3. [Authentication](#)
4. [Sessions](#)
5. [Activities](#)
6. [Sources](#)
7. [Types Reference](#)

Quickstart

Fuente: <https://jules.google/docs/api/reference/>

Quickstart

The Jules REST API lets you programmatically access Jules's capabilities to automate and enhance your software development lifecycle. You can use the API to create custom workflows, automate tasks like bug fixing and code reviews, and embed Jules's intelligence directly into the tools you use every day, such as Slack, Linear, and GitHub.

The Jules REST API is in an alpha release, which means it is experimental. Be aware that we may change specifications, API keys, and definitions as we work toward stabilization. In the future, we plan to maintain at least one stable and one experimental version.

Authentication

[Section titled "Authentication"](#)

To get started with the Jules REST API, you'll need an API key.

Generate Your API Key

[Section titled "Generate Your API Key"](#)

In the Jules web app, go to the [Settings](#) page to create a new API key. You can have at most 3 API keys at a time.

 API key Create key

Your API keys are listed below.

 **Abuse auto-detection on**
For your protection, we automatically delete any API keys found to be publicly exposed to prevent abuse.

API key	Created at	Last used
No keys to display		

Remember to use your API keys securely. Don't share or embed them in public code.
[Learn more here](#)

Use Your API Key

[Section titled "Use Your API Key"](#)

To authenticate your requests, pass the API key in the `X-Goog-API-Key` header of your API calls.

Keep your API keys secure. Don't share them or embed them in public code. For your protection, any API keys found to be publicly exposed will be [automatically disabled](#) to prevent abuse.

API concepts

[Section titled "API concepts"](#)

The Jules REST API is built around a few core resources. Understanding these will help you use the API effectively.

- **Source** — An input source for the agent (e.g., a GitHub repository). Before using a source using the API, you must first [install the Jules GitHub app](#) through the Jules web app.
- **Session** — A continuous unit of work within a specific context, similar to a chat session. A session is initiated with a prompt and a source.
- **Activity** — A single unit of work within a Session. A Session contains multiple activities from both the user and the agent, such as generating a plan, sending a message, or updating progress.

Your first API call

[Section titled "Your first API call"](#)

We'll walk through creating your first session with the Jules REST API using curl.

1. List your available sources

[Section titled "List your available sources"](#)

First, you need to find the name of the source you want to work with (e.g., your GitHub repo). This command will return a list of all sources you have connected to Jules.

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sources
```

The response will look something like this:

```
{
  "sources": [
    {
      "name": "sources/github/bobalover/boba",
      "id": "github/bobalover/boba",
      "githubRepo": {
        "owner": "bobalover",
        "repo": "boba"
      }
    }
  ],
  "nextPageToken": "github/bobalover/boba-web"
}
```

2. Create a new session

[Section titled "Create a new session"](#)

Now, create a new session. You'll need the source name from the previous step. This request tells Jules to create a boba app in the specified repository.

```
curl 'https://jules.googleapis.com/v1alpha/sessions' \
-X POST \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $JULES_API_KEY" \
-d '{
  "prompt": "Create a boba app!",
  "sourceContext": {
    "source": "sources/github/bobalover/boba",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  },
  "automationMode": "AUTO_CREATE_PR",
  "title": "Boba App"
}'
```

The `automationMode` field is optional. By default, no PR will be automatically created.

The immediate response will look something like this:

```
{
  "name": "sessions/31415926535897932384",
  "id": "31415926535897932384",
  "title": "Boba App",
  "sourceContext": {
    "source": "sources/github/bobalover/boba",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  },
  "prompt": "Create a boba app!"
}
```

You can poll the latest session information using `GetSession` or `ListSessions`. For example, if a PR was automatically created, you can see the PR in the session output:

```
{
  "name": "sessions/31415926535897932384",
  "id": "31415926535897932384",
  "title": "Boba App",
  "sourceContext": {
    "source": "sources/github/bobalover/boba",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  },
  "prompt": "Create a boba app!",
  "outputs": [
    {
      "pullRequest": {
        "url": "https://github.com/bobalover/boba/pull/35",
        "title": "Create a boba app",
        "description": "This change adds the initial implementation of a boba app."
      }
    }
  ]
}
```

By default, sessions created through the API will have their plans automatically approved. If you want to create a session that requires explicit plan approval, set the `requirePlanApproval` field to `true`.

3. List sessions

[Section titled "List sessions"](#)

You can list your sessions as follows:

```
curl 'https://jules.googleapis.com/v1alpha/sessions?pageSize=5' \
-H "x-goog-api-key: $JULES_API_KEY"
```

4. Approve a plan

[Section titled "Approve a plan"](#)

If your session requires explicit plan approval, you can approve the latest plan as follows:

```
curl 'https://jules.googleapis.com/v1alpha/sessions/SESSION_ID:approvePlan' \
-X POST \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $JULES_API_KEY"
```

5. Interact with the agent

[Section titled “Interact with the agent”](#)

To list activities in a session:

```
curl 'https://jules.googleapis.com/v1alpha/sessions/SESSION_ID/activities?pageSize=30' \
-H "x-goog-api-key: $JULES_API_KEY"
```

To send a message to the agent:

```
curl 'https://jules.googleapis.com/v1alpha/sessions/SESSION_ID:sendMessage' \
-X POST \
-H "Content-Type: application/json" \
-H "x-goog-api-key: $JULES_API_KEY" \
-d '{
  "prompt": "Can you make the app corgi themed?"
}'
```

The response will be empty because the agent will send its response in the next activity. To see the agent's response, list the activities again.

Next steps

[Section titled “Next steps”](#)

[API Reference](#) Complete documentation for all Jules REST API endpoints, request/response formats, and data types.

API Reference

Fuente: <https://jules.google/docs/api/reference/overview>

API Reference

The Jules REST API allows you to programmatically create and manage coding sessions, monitor progress, and retrieve results. This reference documents all available endpoints, request/response formats, and data types.

Base URL

[Section titled "Base URL"](#)

All API requests should be made to:

```
https://jules.googleapis.com/v1alpha
```

Authentication

[Section titled "Authentication"](#)

The Jules REST API uses API keys for authentication. Get your API key from jules.google.com/settings.

```
curl -H "x-goog-api-key: $JULES_API_KEY" \  
https://jules.googleapis.com/v1alpha/sessions
```

[Authentication Guide](#) Detailed setup instructions for API key authentication.

Endpoints

[Section titled "Endpoints"](#)

[Sessions](#) Create and manage coding sessions. Sessions represent a unit of work where Jules executes tasks on your codebase.

[Activities](#) Monitor session progress through activities. Each activity represents an event like plan generation, messages, or completion.

[Sources](#) List and retrieve connected repositories. Sources represent GitHub repositories that Jules can work with.

[Types](#) Reference for all data types used in the API including Session, Activity, Plan, Artifact, and more.

Common Patterns

[Section titled "Common Patterns"](#)

Pagination

[Section titled "Pagination"](#)

List endpoints support pagination using `pageSize` and `pageToken` parameters:



```
# First page
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sessions?pageSize=10"

# Next page (using token from previous response)
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sessions?pageSize=10&pageToken=NEXT_PAGE_TOKEN"
```

Resource Names

[Section titled "Resource Names"](#)

Resources use hierarchical names following Google API conventions:

- Sessions: `sessions/{sessionId}`
- Activities: `sessions/{sessionId}/activities/{activityId}`
- Sources: `sources/{sourceId}`

Error Handling

[Section titled "Error Handling"](#)

The API returns standard HTTP status codes:

Status	Description
200	Success
400	Bad request - invalid parameters
401	Unauthorized - invalid or missing token
403	Forbidden - insufficient permissions
404	Not found - resource doesn't exist
429	Rate limited - too many requests
500	Server error

Error responses include a JSON body with details:

```
{
  "error": {
    "code": 400,
    "message": "Invalid session ID format",
    "status": "INVALID_ARGUMENT"
  }
}
```

Authentication

Fuente: <https://jules.google/docs/api/reference/authentication>

Authentication

The Jules REST API uses API keys for authentication. You'll need a valid API key to make API requests.

Getting Your API Key

[Section titled "Getting Your API Key"](#)

1. Go to jules.google.com/settings
2. Find the **API Key** section
3. Click **Generate API Key** (or copy your existing key)
4. Store the key securely — it won't be shown again

Keep your API key secret. Don't commit it to version control or share it publicly.

Using Your API Key

[Section titled "Using Your API Key"](#)

Include the API key in the `x-goog-api-key` header with every request:

```
curl -H "x-goog-api-key: YOUR_API_KEY" \
  https://jules.googleapis.com/v1alpha/sessions
```

Environment Variable (Recommended)

[Section titled "Environment Variable \(Recommended\)"](#)

Store your API key in an environment variable:

```
export JULES_API_KEY="your-api-key-here"
```

Then use it in requests:

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sessions
```

Example: Create a Session

[Section titled "Example: Create a Session"](#)

```
curl -X POST \
-H "x-goog-api-key: $JULES_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "prompt": "Add unit tests for the utils module",
  "sourceContext": {
    "source": "sources/github-owner-repo",
    "githubRepoContext": {
      "startingBranch": "main"
    }
  }
}' \
https://jules.googleapis.com/v1alpha/sessions
```

Example: List Sessions

[Section titled "Example: List Sessions"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sessions
```

Troubleshooting

[Section titled "Troubleshooting"](#)

"API key not valid"

[Section titled ""API key not valid""](#)

- Verify you copied the entire key without extra spaces
- Check that the key hasn't been revoked in [settings](#)
- Generate a new key if needed

"Permission denied"

[Section titled ""Permission denied""](#)

- Verify your account has access to Jules
- Check that you have access to the requested resources (sessions, sources)

“Quota exceeded”

Section titled ““Quota exceeded””

- You may have hit rate limits

Sessions

Fuente: <https://jules.google/docs/api/reference/sessions>

Sessions

Sessions are the core resource in the Jules REST API. A session represents a unit of work where Jules executes a coding task on your repository.

Create a Session

[Section titled "Create a Session"](#)

POST `/v1alpha/sessions`

Creates a new session to start a coding task.

Request Body

[Section titled "Request Body"](#)

`prompt` required string

The task description for Jules to execute.

`title` string

Optional title for the session. If not provided, the system will generate one.

`sourceContext` [SourceContext](#)

The source repository and branch context for this session. Optional for repoless sessions.

`requirePlanApproval` boolean

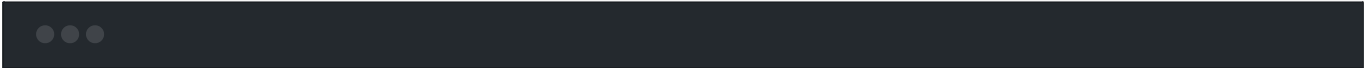
If true, plans require explicit approval before execution. If not set, plans are auto-approved.

`automationMode` string

Automation mode. Use 'AUTO_CREATE_PR' to automatically create pull requests when code changes are ready.

Example Request

[Section titled "Example Request"](#)



```
curl -X POST \
  -H "x-goog-api-key: $JULES_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Add comprehensive unit tests for the authentication module",
    "title": "Add auth tests",
    "sourceContext": {
      "source": "sources/github-myorg-myrepo",
      "githubRepoContext": {
        "startingBranch": "main"
      }
    },
    "requirePlanApproval": true
  }' \
  https://jules.googleapis.com/v1alpha/sessions
```

Response

[Section titled "Response"](#)

Returns the created [Session](#) object:

```
{
  "name": "sessions/1234567",
  "id": "abc123",
  "prompt": "Add comprehensive unit tests for the authentication module",
  "title": "Add auth tests",
  "state": "QUEUED",
  "url": "https://jules.google.com/session/abc123",
  "createTime": "2024-01-15T10:30:00Z",
  "updateTime": "2024-01-15T10:30:00Z"
}
```

List Sessions

[Section titled "List Sessions"](#)

GET `/v1alpha/sessions`

Lists all sessions for the authenticated user.

Query Parameters

[Section titled "Query Parameters"](#)

`pageSize` integer query

Number of sessions to return (1-100). Defaults to 30.

`pageToken` string query

Page token from a previous ListSessions response.

Example Request

[Section titled "Example Request"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sessions?pageSize=10"
```

Response

[Section titled "Response"](#)

```
{
  "sessions": [
    {
      "name": "sessions/1234567",
      "id": "abc123",
      "title": "Add auth tests",
      "state": "COMPLETED",
      "createTime": "2024-01-15T10:30:00Z",
      "updateTime": "2024-01-15T11:45:00Z"
    }
  ],
  "nextPageToken": "eyJvZmZzZXQiOjEwEwfQ=="
}
```

Get a Session

[Section titled "Get a Session"](#)

GET `/v1alpha/sessions/{sessionId}`

Retrieves a single session by ID.

Path Parameters

[Section titled "Path Parameters"](#)

`name` required string path

The resource name of the session. Format: sessions/{session}

Pattern: `^sessions/[/\stimes]+$`

Example Request

[Section titled "Example Request"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
  https://jules.googleapis.com/v1alpha/sessions/1234567
```

Response

[Section titled "Response"](#)

Returns the full [Session](#) object including outputs if the session has completed:

```
{
  "name": "sessions/1234567",
  "id": "abc123",
  "prompt": "Add comprehensive unit tests for the authentication module",
  "title": "Add auth tests",
  "state": "COMPLETED",
  "url": "https://jules.google.com/session/abc123",
  "createTime": "2024-01-15T10:30:00Z",
  "updateTime": "2024-01-15T11:45:00Z",
  "outputs": [
    {
      "pullRequest": {
        "url": "https://github.com/myorg/myrepo/pull/42",
        "title": "Add auth tests",
        "description": "Added unit tests for authentication module"
      }
    }
  ]
}
```

Delete a Session

[Section titled "Delete a Session"](#)

DELETE `/v1alpha/sessions/{sessionId}`

Deletes a session.

Path Parameters

[Section titled "Path Parameters"](#)

`name` required string path

The resource name of the session to delete. Format: sessions/{session}

Pattern: `^sessions/[^\s/]+$`

Example Request

[Section titled "Example Request"](#)

```
curl -X DELETE \
  -H "x-goog-api-key: $JULES_API_KEY" \
  https://jules.googleapis.com/v1alpha/sessions/1234567
```

Response

[Section titled "Response"](#)

Returns an empty response on success.

Send a Message

[Section titled "Send a Message"](#)

POST `/v1alpha/sessions/{sessionId}:sendMessage`

Sends a message from the user to an active session.

Use this endpoint to provide feedback, answer questions, or give additional instructions to Jules during an active session.

Path Parameters

[Section titled "Path Parameters"](#)

`session` required string path

The resource name of the session. Format: sessions/{session}

Pattern: `^sessions/[/]+[extract_itex]`

Request Body

[Section titled "Request Body"](#)

`prompt` required string

The message to send to the session.

Example Request

[Section titled "Example Request"](#)



```
curl -X POST \
  -H "x-goog-api-key: [extract_itex]JULES_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Please also add integration tests for the login flow"
  }' \
  https://jules.googleapis.com/v1alpha/sessions/1234567:sendMessage
```

Response

[Section titled "Response"](#)

Returns an empty [SendMessageResponse](#) on success.

Approve a Plan

[Section titled "Approve a Plan"](#)

POST `/v1alpha/sessions/{sessionId}:approvePlan`

Approves a pending plan in a session.

This endpoint is only needed when `requirePlanApproval` was set to `true` when creating the session.

Path Parameters

[Section titled "Path Parameters"](#)

`session` required string path

The resource name of the session. Format: sessions/{session}

Pattern: `^sessions/[^/]+$`

Example Request

[Section titled "Example Request"](#)



```
curl -X POST \
  -H "x-goog-api-key: $JULES_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{}' \
  https://jules.googleapis.com/v1alpha/sessions/1234567:approvePlan
```

Response

[Section titled "Response"](#)

Returns an empty [ApprovePlanResponse](#) on success.

Session States

[Section titled "Session States"](#)

Sessions progress through the following states:

State	Description
QUEUED	Session is waiting to be processed
PLANNING	Jules is analyzing the task and creating a plan
AWAITING_PLAN_APPROVAL	Plan is ready and waiting for user approval
AWAITING_USER_FEEDBACK	Jules needs additional input from the user
IN_PROGRESS	Jules is actively working on the task
PAUSED	Session is paused
COMPLETED	Task completed successfully
FAILED	Task failed to complete

Activities

Fuente: <https://jules.google/docs/api/reference/activities>

Activities

Activities represent events that occur during a session. Use the Activities API to monitor progress, retrieve messages, and access artifacts like code changes.

List Activities

[Section titled "List Activities"](#)

GET `/v1alpha/sessions/{sessionId}/activities`

Lists all activities for a session.

Path Parameters

[Section titled "Path Parameters"](#)

`parent` required string path

The parent session. Format: sessions/{session}

Pattern: `^sessions/[^/]+$`

Query Parameters

[Section titled "Query Parameters"](#)

`pageSize` integer query

Number of activities to return (1-100). Defaults to 50.

`pageToken` string query

Page token from a previous ListActivities response.

Example Requests

[Section titled "Example Requests"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sessions/1234567/activities?pageSize=20"
```

```
# Define the endpoint to prevent overflow
ENDPOINT="https://jules.googleapis.com/v1alpha/sessions/1234567/activities"
TIMESTAMP="2026-01-17T00:03:53.137240Z"

curl -H "x-goog-api-key: $JULES_API_KEY" \
    "$ENDPOINT?createTime=$TIMESTAMP"
```

Response

Section titled "Response"

```
{
  "activities": [
    {
      "name": "sessions/1234567/activities/act1",
      "id": "act1",
      "originator": "system",
      "description": "Session started",
      "createTime": "2024-01-15T10:30:00Z"
    },
    {
      "name": "sessions/1234567/activities/act2",
      "id": "act2",
      "originator": "agent",
      "description": "Plan generated",
      "planGenerated": {
        "plan": {
          "id": "plan1",
          "steps": [
            {
              "id": "step1",
              "index": 0,
              "title": "Analyze existing code",
              "description": "Review the authentication module structure"
            },
            {
              "id": "step2",
              "index": 1,
              "title": "Write unit tests",
              "description": "Create comprehensive test coverage"
            }
          ]
        },
        "createTime": "2024-01-15T10:31:00Z"
      }
    },
    {
      "createTime": "2024-01-15T10:31:00Z"
    }
  ],
  "nextPageToken": "eyJvZmZzZXQiOiIwIiwfQ=="
}
```

Get an Activity

[Section titled "Get an Activity"](#)

GET `/v1alpha/sessions/{sessionId}/activities/{activityId}`

Retrieves a single activity by ID.

Path Parameters

[Section titled "Path Parameters"](#)

`name` required string path

The resource name of the activity. Format: sessions/{session}/activities/{activity}

Pattern: `^sessions/[^/]+/activities/[^/]+$`

Example Request

[Section titled "Example Request"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sessions/1234567/activities/act2
```

Response

[Section titled "Response"](#)

Returns the full [Activity](#) object:

```
{
  "name": "sessions/1234567/activities/act2",
  "id": "act2",
  "originator": "agent",
  "description": "Code changes ready",
  "createTime": "2024-01-15T11:00:00Z",
  "artifacts": [
    {
      "changeSet": {
        "source": "sources/github-myorg-myrepo",
        "gitPatch": {
          "baseCommitId": "a1b2c3d4",
          "unidiffPatch": "diff --git a/tests/auth.test.js...",
          "suggestedCommitMessage": "Add unit tests for authentication module"
        }
      }
    }
  ]
}
```

Activity Types

[Section titled "Activity Types"](#)

Activities have different types based on what occurred. Each activity will have exactly one of these event fields populated:

Plan Generated

[Section titled "Plan Generated"](#)

Indicates Jules has created a plan for the task:

```
{
  "planGenerated": {
    "plan": {
      "id": "plan1",
      "steps": [
        { "id": "step1", "index": 0, "title": "Step title", "description": "Details" }
      ],
      "createTime": "2024-01-15T10:31:00Z"
    }
  }
}
```

Plan Approved

[Section titled "Plan Approved"](#)

Indicates a plan was approved (by user or auto-approved):

```
{
  "planApproved": {
    "planId": "plan1"
  }
}
```

User Messaged

[Section titled "User Messaged"](#)

A message from the user:

```
{
  "userMessaged": {
    "userMessage": "Please also add integration tests"
  }
}
```

Agent Messaged

[Section titled "Agent Messaged"](#)

A message from Jules:

```
{
  "agentMessaged": {
    "agentMessage": "I've completed the unit tests. Would you like me to add integrati
on tests as well?"
  }
}
```

Progress Updated

[Section titled "Progress Updated"](#)

A status update during execution:

```
{
  "progressUpdated": {
    "title": "Writing tests",
    "description": "Creating test cases for login functionality"
  }
}
```

Session Completed

[Section titled "Session Completed"](#)

The session finished successfully:

```
{
  "sessionCompleted": {}
}
```

Session Failed

[Section titled "Session Failed"](#)

The session encountered an error:

```
{
  "sessionFailed": {
    "reason": "Unable to install dependencies"
  }
}
```

Artifacts

[Section titled "Artifacts"](#)

Activities may include artifacts—outputs produced during execution:

Code Changes (ChangeSet)

[Section titled "Code Changes \(ChangeSet\)"](#)

```
{
  "artifacts": [
    {
      "changeSet": {
        "source": "sources/github-myorg-myrepo",
        "gitPatch": {
          "baseCommitId": "a1b2c3d4e5f6",
          "unidiffPatch": "diff --git a/src/auth.js b/src/auth.js\n...",
          "suggestedCommitMessage": "Add authentication tests"
        }
      }
    }
  ]
}
```

Bash Output

Section titled "Bash Output"

```
{
  "artifacts": [
    {
      "bashOutput": {
        "command": "npm test",
        "output": "All tests passed (42 passing)",
        "exitCode": 0
      }
    }
  ]
}
```

Media

Section titled "Media"

```
{
  "artifacts": [
    {
      "media": {
        "mimeType": "image/png",
        "data": "base64-encoded-data..."
      }
    }
  ]
}
```

Sources

Fuente: <https://jules.google/docs/api/reference/sources>

Sources

Sources represent repositories connected to Jules. Currently, Jules supports GitHub repositories. Use the Sources API to list available repositories and get details about specific sources.

Sources are created when you connect a GitHub repository to Jules through the web interface. The API currently only supports reading sources, not creating them.

List Sources

[Section titled "List Sources"](#)

GET `/v1alpha/sources`

Lists all sources (repositories) connected to your account.

Query Parameters

[Section titled "Query Parameters"](#)

`pageSize` integer query

Number of sources to return (1-100). Defaults to 30.

`pageToken` string query

Page token from a previous ListSources response.

`filter` string query

Filter expression based on AIP-160. Example: 'name=sources/source1 OR name=sources/source2'

Example Request

[Section titled "Example Request"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sources?pageSize=10"
```

Response

[Section titled "Response"](#)

```
{
  "sources": [
    {
      "name": "sources/github-myorg-myrepo",
      "id": "github-myorg-myrepo",
      "githubRepo": {
        "owner": "myorg",
        "repo": "myrepo",
        "isPrivate": false,
        "defaultBranch": {
          "displayName": "main"
        }
      },
      "branches": [
        { "displayName": "main" },
        { "displayName": "develop" },
        { "displayName": "feature/auth" }
      ]
    },
    {
      "name": "sources/github-myorg-another-repo",
      "id": "github-myorg-another-repo",
      "githubRepo": {
        "owner": "myorg",
        "repo": "another-repo",
        "isPrivate": true,
        "defaultBranch": {
          "displayName": "main"
        }
      },
      "branches": [
        { "displayName": "main" }
      ]
    }
  ],
  "nextPageToken": "eyJvZmZzZXQiOjEwfQ=="
}
```

Filtering

Section titled "Filtering"

Use the `filter` parameter to retrieve specific sources:

```
# Get a specific source
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sources?filter=name%3Dsources%2Fgithub-myorg-m
yrepo"

# Get multiple sources
curl -H "x-goog-api-key: $JULES_API_KEY" \
  "https://jules.googleapis.com/v1alpha/sources?filter=name%3Dsources%2Fsource1%20OR%2
0name%3Dsources%2Fsource2"
```

Get a Source

[Section titled "Get a Source"](#)

GET `/v1alpha/sources/{sourceId}`

Retrieves a single source by ID.

Path Parameters

[Section titled "Path Parameters"](#)

`name` required string path

The resource name of the source. Format: sources/{source}

Pattern: `^sources/.*$`

Example Request

[Section titled "Example Request"](#)

```
curl -H "x-goog-api-key: $JULES_API_KEY" \
https://jules.googleapis.com/v1alpha/sources/github-myorg-myrepo
```

Response

[Section titled "Response"](#)

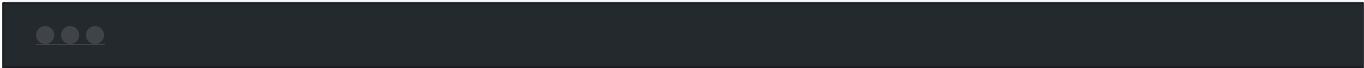
Returns the full [Source](#) object:

```
{
  "name": "sources/github-myorg-myrepo",
  "id": "github-myorg-myrepo",
  "githubRepo": {
    "owner": "myorg",
    "repo": "myrepo",
    "isPrivate": false,
    "defaultBranch": {
      "displayName": "main"
    }
  },
  "branches": [
    { "displayName": "main" },
    { "displayName": "develop" },
    { "displayName": "feature/auth" },
    { "displayName": "feature/tests" }
  ]
}
```

Using Sources with Sessions

[Section titled "Using Sources with Sessions"](#)

When creating a session, reference a source using its resource name in the `sourceContext`:



```
curl -X POST \
  -H "x-goog-api-key: $JULES_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Add unit tests for the auth module",
    "sourceContext": {
      "source": "sources/github-myorg-myrepo",
      "githubRepoContext": {
        "startingBranch": "develop"
      }
    }
  }' \
  https://jules.googleapis.com/v1alpha/sessions
```

Use the List Sources endpoint to discover available source names, then use the Get Source endpoint to see available branches before creating a session.

Types Reference

Fuente: <https://jules.google/docs/api/reference/types>

Types Reference

This page documents all data types used in the Jules REST API.

Core Resources

[Section titled "Core Resources"](#)

Session

[Section titled "Session"](#)

A session represents a unit of work where Jules executes a coding task.

Session

A session is a contiguous amount of work within the same context.

name string

Output only. The full resource name (e.g., 'sessions/{session}').

id string

Output only. The session ID.

prompt required string

The task description for Jules to execute.

title string

Optional title. If not provided, the system generates one.

state [SessionState](#)

Output only. Current state of the session.

url string

Output only. URL to view the session in the Jules web app.

sourceContext required [SourceContext](#)

The source repository and branch context.

requirePlanApproval boolean

Input only. If true, plans require explicit approval.

`automationMode` [AutomationMode](#)

Input only. Automation mode for the session.

`outputs` [SessionOutput](#) []

Output only. Results of the session (e.g., pull requests).

`createTime` string (google-datetime)

Output only. When the session was created.

`updateTime` string (google-datetime)

Output only. When the session was last updated.

SessionState

[Section titled "SessionState"](#)

Enum representing the current state of a session:

Value	Description
<code>STATE_UNSPECIFIED</code>	State is unspecified
<code>QUEUED</code>	Session is waiting to be processed
<code>PLANNING</code>	Jules is creating a plan
<code>AWAITING_PLAN_APPROVAL</code>	Plan is ready for user approval
<code>AWAITING_USER_FEEDBACK</code>	Jules needs user input
<code>IN_PROGRESS</code>	Jules is actively working
<code>PAUSED</code>	Session is paused
<code>FAILED</code>	Session failed
<code>COMPLETED</code>	Session completed successfully

AutomationMode

[Section titled "AutomationMode"](#)

Enum for session automation settings:

Value	Description
<code>AUTOMATION_MODE_UNSPECIFIED</code>	No automation (default)
<code>AUTO_CREATE_PR</code>	Automatically create a pull request when code changes are ready

Activity

[Section titled "Activity"](#)

An activity represents a single event within a session.

Activity

An activity is a single unit of work within a session.

`name` string

The full resource name (e.g., 'sessions/{session}/activities/{activity}').

`id` string

Output only. The activity ID.

`originator` string

The entity that created this activity ('user', 'agent', or 'system').

`description` string

Output only. A description of this activity.

`createTime` string (google-datetime)

Output only. When the activity was created.

`artifacts` [Artifact](#) []

Output only. Artifacts produced by this activity.

`planGenerated` [PlanGenerated](#)

A plan was generated.

`planApproved` [PlanApproved](#)

A plan was approved.

`userMessaged` [UserMessaged](#)

The user posted a message.

`agentMessaged` [AgentMessaged](#)

Jules posted a message.

`progressUpdated` [ProgressUpdated](#)

A progress update occurred.

`sessionCompleted` [SessionCompleted](#)

The session completed.

`sessionFailed` [SessionFailed](#)

The session failed.

Source

[Section titled "Source"](#)

A source represents a connected repository.

Source

An input source of data for a session.

`name` string

The full resource name (e.g., 'sources/{source}').

`id` string

Output only. The source ID.

`githubRepo` [GitHubRepo](#)

GitHub repository details.

Plans

[Section titled "Plans"](#)

Plan

[Section titled "Plan"](#)

Plan

A sequence of steps that Jules will take to complete the task.

`id` string

Output only. Unique ID for this plan within a session.

`steps` [PlanStep](#) []

Output only. The steps in the plan.

`createTime` string (google-datetime)

Output only. When the plan was created.

PlanStep

[Section titled "PlanStep"](#)

PlanStep

A single step in a plan.

`id` string

Output only. Unique ID for this step within a plan.

`index` integer (int32)

Output only. 0-based index in the plan.

`title` string

Output only. The title of the step.

`description` string

Output only. Detailed description of the step.

Artifacts

[Section titled "Artifacts"](#)

Artifact

[Section titled "Artifact"](#)

Artifact

A single unit of data produced by an activity.

`changeSet` [ChangeSet](#)

Code changes produced.

`bashOutput` [BashOutput](#)

Command output produced.

`media` [Media](#)

Media file produced (e.g., image, video).

ChangeSet

[Section titled "ChangeSet"](#)

ChangeSet

A set of changes to be applied to a source.

`source` string

The source this change set applies to. Format: sources/{source}

`gitPatch` [GitPatch](#)

The patch in Git format.

GitPatch

[Section titled "GitPatch"](#)

GitPatch

A patch in Git format.

`baseCommitId` string

The commit ID the patch should be applied to.

`unidiffPatch` string

The patch in unified diff format.

`suggestedCommitMessage` string

A suggested commit message for the patch.

BashOutput

[Section titled "BashOutput"](#)

BashOutput

Output from a bash command.

`command` string

The bash command that was executed.

`output` string

Combined stdout and stderr output.

`exitCode` integer (int32)

The exit code of the command.

Media

[Section titled "Media"](#)

Media

A media file output.

`contentType` string

The MIME type of the media (e.g., 'image/png').

`data` string (byte)

Base64-encoded media data.

GitHub Types

[Section titled "GitHub Types"](#)

GitHubRepo

[Section titled "GitHubRepo"](#)

GitHubRepo

A GitHub repository.

owner string

The repository owner (user or organization).

repo string

The repository name.

isPrivate boolean

Whether the repository is private.

defaultBranch [GitHubBranch](#)

The default branch.

branches [GitHubBranch](#) []

List of active branches.

GitHubBranch

[Section titled "GitHubBranch"](#)

GitHubBranch

A GitHub branch.

displayName string

The branch name.

GitHubRepoContext

[Section titled "GitHubRepoContext"](#)

GitHubRepoContext

Context for using a GitHub repo in a session.

startingBranch required string

The branch to start the session from.

Context Types

[Section titled "Context Types"](#)

SourceContext

[Section titled "SourceContext"](#)

SourceContext

Context for how to use a source in a session.

`source` required string

The source resource name. Format: sources/{source}

`githubRepoContext` [GitHubRepoContext](#)

Context for GitHub repositories.

Output Types

[Section titled "Output Types"](#)

SessionOutput

[Section titled "SessionOutput"](#)

SessionOutput

An output of a session.

`pullRequest` [PullRequest](#)

A pull request created by the session.

PullRequest

[Section titled "PullRequest"](#)

PullRequest

A pull request.

`url` string

The URL of the pull request.

`title` string

The title of the pull request.

`description` string

The description of the pull request.

Activity Event Types

[Section titled "Activity Event Types"](#)

PlanGenerated

[Section titled "PlanGenerated"](#)

PlanGenerated

A plan was generated.

`plan` [Plan](#)

The generated plan.

PlanApproved

[Section titled "PlanApproved"](#)

PlanApproved

A plan was approved.

`planId` string

The ID of the approved plan.

UserMessaged

[Section titled "UserMessaged"](#)

UserMessaged

The user posted a message.

`userMessage` string

The message content.

AgentMessaged

[Section titled "AgentMessaged"](#)

AgentMessaged

Jules posted a message.

`agentMessage` string

The message content.

ProgressUpdated

[Section titled "ProgressUpdated"](#)

ProgressUpdated

A progress update occurred.

title string

The title of the update.

description string

Details about the progress.

SessionCompleted

[Section titled "SessionCompleted"](#)

SessionCompleted

The session completed successfully.

No additional properties.

SessionFailed

[Section titled "SessionFailed"](#)

SessionFailed

The session failed.

reason string

The reason for the failure.

Request/Response Types

[Section titled "Request/Response Types"](#)

SendMessageRequest

[Section titled "SendMessageRequest"](#)

SendMessageRequest

Request to send a message to a session.

prompt required string

The message to send.

SendMessageResponse

[Section titled "SendMessageResponse"](#)

SendMessageResponse

Response from sending a message.

Empty response on success.

ApprovePlanRequest

[Section titled "ApprovePlanRequest"](#)

ApprovePlanRequest

Request to approve a plan.

Empty request body.

ApprovePlanResponse

[Section titled "ApprovePlanResponse"](#)

ApprovePlanResponse

Response from approving a plan.

Empty response on success.

ListSessionsResponse

[Section titled "ListSessionsResponse"](#)

ListSessionsResponse

Response from listing sessions.

`sessions` [Session](#) []

The list of sessions.

`nextPageToken` string

Token for the next page of results.

ListActivitiesResponse

[Section titled "ListActivitiesResponse"](#)

ListActivitiesResponse

Response from listing activities.

`activities` [Activity](#) []

The list of activities.

`nextPageToken` string

Token for the next page of results.

ListSourcesResponse

[Section titled "ListSourcesResponse"](#)

ListSourcesResponse

Response from listing sources.

`sources` [Source](#) []

The list of sources.

`nextPageToken` string

Token for the next page of results.